

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO4 ASSESSMENT TOOL 1 – Test Case Design Assignment

Course Code:	CSA10	Course Title:	Software Engineering
CO Assessed:	CO4	Assessment:	Assessment Tool 1 – Test Case Design Assignment
Weightage:	20%	Total Marks:	25
CO4 Statement:	Implement robust testing, quality assurance, and DevOps practices, emphasizing security, reliability, and ethics		
Student Name:	M.HASHETHA	Reg. No.:	192521162
Date:	18.08.2026	Duration:	60 minutes

Code of Conduct

I M.HASHETHA(192521162) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: M.HASHETHA

Annexure A – Test Case Design Assignment

Instructions to Students:

Each student is assigned an individual problem statement. Based on the assigned problem statement, **design and document comprehensive test cases** to verify the correctness, functionality, reliability, and usability of the proposed system.

Your test case design should include:

1. Identify the **functional and non-functional requirements** to be tested.
2. Identify the **test scenarios** for the assigned problem statement.
3. Prepare detailed **test cases** covering normal, boundary, invalid, and exceptional conditions.
4. Include **Test Case ID, Test Scenario, Test Steps, Test Data, Expected Result, Actual Result, and Pass/Fail Status**.
5. Apply appropriate **test design techniques** such as Equivalence Partitioning, Boundary Value Analysis, Decision Table, or State Transition Testing wherever applicable.
6. Ensure that the test cases provide adequate **requirement and functional coverage** for the assigned system.

Deliverable: Submit a Test Case Design document containing the identified scenarios, test cases, test data, expected results, and test coverage based on the individual problem statement.

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Requirement & Test Scenario Identification(15%)	Clearly identifies all relevant functional/non-functional requirements and comprehensive test scenarios	Identifies most requirements and scenarios	Identifies basic requirements and scenarios	Requirements/scenarios are incomplete or unclear
Test Case Design & Completeness(25%)	Comprehensive test cases covering normal, boundary, invalid, and exceptional conditions	Covers most important conditions with minor gaps	Covers basic conditions but misses several important cases	Test cases are very limited or incomplete
Test Data & Expected Results(20%)	Appropriate test data with precise, measurable expected results for every test case	Mostly appropriate test data and expected results	Some test data/results are unclear or incomplete	Test data and expected results are largely missing/incorrect
Application of Test Design Techniques(15%)	Correctly applies multiple techniques such as Equivalence Partitioning, Boundary Value Analysis, Decision Table, and State Transition	Correctly applies at least two relevant techniques	Applies one technique with limited justification	Techniques are not applied or incorrectly applied
Traceability & Test Coverage(15%)	Excellent mapping between requirements, scenarios, and test cases with strong coverage	Good traceability and coverage with minor gaps	Partial traceability and moderate coverage	Poor/no traceability and inadequate coverage
Documentation & Presentation(10%)	Well-structured, clear, professional, consistent, and easy to understand	Well-organized with minor presentation issues	Adequately organized but has some clarity/formatting issues	Poorly organized, unclear, or incomplete documentation

Criteria	Mark
Requirement & Test Scenario Identification (15%)	/5
Test Case Design & Completeness (25%)	/4
Test Data & Expected Results (20%)	/4
Application of Test Design Techniques (15%)	/4
Traceability & Test Coverage (15%)	/4
Documentation & Presentation (10%)	/4
TOTAL	/25

AI-Powered Smart Court Case Management and Judicial Analytics Platform

Problem Statement:

Courts often experience delays in case processing due to manual documentation, inefficient scheduling, and limited analytical tools. Existing systems provide minimal support for case prioritization and judicial workload management. An AI-powered judicial platform should automate

document classification, schedule hearings intelligently, analyze case trends, predict case durations,

and provide dashboards for judicial administration. The system should improve court efficiency while ensuring transparency and accessibility.



Question 1: Identify the Functional and Non-Functional Requirements to be Tested

Answer:

The **AI-Powered Smart Court Case Management and Judicial Analytics Platform** is designed to digitize court case management, automate legal document classification, optimize hearing schedules, predict case completion timelines, analyze judicial workload, and improve transparency. Testing the functional and non-functional requirements is essential to ensure that the system operates correctly, securely, reliably, and efficiently.

1. Functional Requirements

1. **User Authentication and Authorization:** The system shall allow authorized users such as judges, court administrators, lawyers, and permitted staff to log in securely and access functions according to their roles.
2. **Case Registration and Management:** The system shall allow authorized users to create, update, view, search, and manage court case records. It should maintain accurate case details throughout the case lifecycle.
3. **Legal Document Classification:** The AI/NLP module shall classify uploaded legal documents into appropriate categories. The system should also handle unsupported, corrupted, or incorrectly formatted documents.
4. **Hearing Scheduling:** The system shall provide suitable hearing dates and times by considering factors such as judge availability, courtroom availability, case priority, and existing schedules.
5. **Case Duration Prediction:** The AI module shall analyze relevant case information and provide an estimated case completion timeline. The prediction should be displayed clearly without being treated as a guaranteed outcome.
6. **Judicial Workload Analysis:** The system shall calculate and display workload information such as pending cases, completed cases, upcoming hearings, and case distribution among judges.
7. **Analytics and Reports:** The system shall generate reports and dashboards related to case backlog, case duration, workload, hearing schedules, and case trends.
8. **Notifications and Alerts:** The system shall provide notifications for upcoming hearings, schedule changes, important case updates, and other authorized alerts.

2. Non-Functional Requirements

1. **Security:** The system shall protect confidential legal information through authentication, authorization, secure data storage, and access control.
2. **Performance:** The system should provide responses within an acceptable time even when multiple users access the platform simultaneously.

3. **Reliability:** The system should operate consistently and prevent loss or corruption of case records and legal documents.
4. **Usability:** The interface should be simple, clear, accessible, and easy to use for judges, court staff, lawyers, and other authorized users.
5. **Scalability:** The platform should support increasing numbers of cases, users, documents, and courts without significant performance degradation.
6. **Availability:** The system should remain available during court working hours and provide appropriate recovery mechanisms in case of system failure.
7. **Accuracy and Transparency:** AI-generated classifications, predictions, and analytics should be explainable where appropriate, clearly identified as AI-assisted, and should not replace independent judicial decision-making.

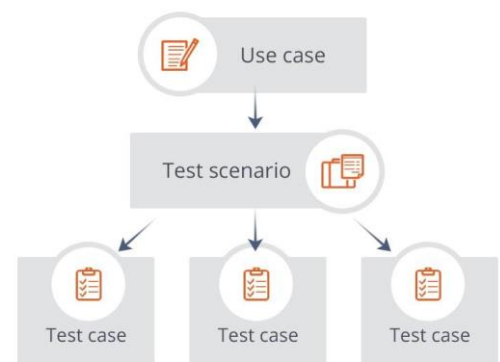
Thus, testing these functional and non-functional requirements will help ensure that the proposed platform is **efficient, secure, reliable, transparent, accessible, and suitable for modern judicial case management.**

Question 2: Identify the Test Scenarios for the Assigned Problem Statement

Answer:

Test scenarios are high-level conditions used to verify whether the **AI-Powered Smart Court Case Management and Judicial Analytics Platform** performs its intended functions correctly. The following test scenarios cover normal, invalid, boundary, and exceptional conditions.

1. **User Registration and Login:** Verify that authorized users can log in using valid credentials and that invalid credentials are rejected with an appropriate error message.
2. **Role-Based Access:** Verify that judges, court administrators, lawyers, and other users can access only the features and information permitted for their respective roles.
3. **Case Registration:** Verify that a new court case can be registered successfully with all required details and that incomplete or invalid case information is rejected.
4. **Case Search and Retrieval:** Verify that users can search for cases using case number, case type, date, status, or other valid search criteria and retrieve accurate results.



5. **Case Record Modification:** Verify that authorized users can update case information and that unauthorized users are prevented from modifying records.
6. **Legal Document Upload:** Verify that valid legal documents can be uploaded, stored, retrieved, and associated with the correct case.
7. **AI-Based Document Classification:** Verify that the AI/NLP module correctly classifies different types of legal documents and handles unclear, unsupported, or corrupted documents appropriately.
8. **Hearing Schedule Management:** Verify that the system schedules hearings based on judge availability, courtroom availability, case priority, and existing schedules without creating conflicts.
9. **Case Duration Prediction:** Verify that the AI module generates an estimated completion timeline using available case information and displays the result correctly.
10. **Judicial Workload Analysis:** Verify that the system accurately calculates pending cases, completed cases, upcoming hearings, and workload distribution among judges.
11. **Dashboard and Analytics:** Verify that dashboards display accurate and updated information regarding case backlog, case trends, workload, and hearing schedules.
12. **Report Generation:** Verify that authorized users can generate and download accurate judicial analytics reports in the supported format.
13. **Notifications and Alerts:** Verify that users receive appropriate notifications for upcoming hearings, schedule changes, and important case updates.
14. **Security Testing:** Verify that confidential case information cannot be accessed by unauthorized users and that user sessions are securely managed.
15. **Performance Testing:** Verify that the platform provides acceptable response times when multiple users access case records, upload documents, or generate reports simultaneously.
16. **Data Integrity and Recovery:** Verify that case records and documents remain accurate after system failures, unexpected shutdowns, or network interruptions.
17. **Boundary Conditions:** Verify system behavior when maximum or minimum permitted values are entered, such as very large documents, maximum search results, or large numbers of cases.
18. **Exception Handling:** Verify that the system provides meaningful error messages and continues operating safely when invalid data, network failures, unavailable services, or AI processing errors occur.
19. **Accessibility and Usability:** Verify that the interface is understandable and accessible to different categories of authorized users.

20. AI Transparency and Ethical Use: Verify that AI-generated classifications, predictions, and analytics are clearly identified as AI-assisted and are presented in a manner that supports, rather than replaces, judicial decision-making.

These test scenarios provide comprehensive coverage of the major functions and quality requirements of the proposed platform. They will form the basis for developing detailed test cases covering **normal, boundary, invalid, and exceptional conditions**.

Question 3: Prepare Detailed Test Cases Covering Normal, Boundary, Invalid, and Exceptional Conditions

Answer:

The following test cases are designed to verify the correctness, functionality, reliability, security, and usability of the **AI-Powered Smart Court Case Management and Judicial Analytics Platform**. The test cases cover normal, boundary, invalid, and exceptional conditions.

Test Case ID	Test Scenario	Test Steps	Test Data	Expected Result	Actual Result
TC01	Valid user login	Enter valid username and password and click Login	Valid credentials	User should be successfully logged in and redirected to the appropriate dashboard	To be tested
TC02	Invalid login	Enter incorrect username/password and click Login	Incorrect credentials	System should reject login and display an appropriate error message	To be tested
TC03	Role-based access	Login using a user account and access restricted functions	Lawyer account attempting administrator function	System should deny unauthorized access	To be tested
TC04	Register new case	Enter all mandatory case details and submit	Valid case number, case type, parties, and date	New case should be created and assigned a unique record	To be tested

Test Case ID	Test Scenario	Test Steps	Test Data	Expected Result	Actual Result
TC05	Incomplete case registration	Leave mandatory fields empty and submit	Missing case details	System should identify missing fields and prevent registration	To be tested
TC06	Search case	Enter a valid case number and search	Existing case number	Correct case information should be displayed	To be tested
TC07	Invalid case search	Search using a non-existing case number	Invalid case number	System should display “No case found” or an equivalent message	To be tested
TC08	Upload legal document	Select a supported document and upload it to a case	Valid PDF legal document	Document should be uploaded successfully and linked to the correct case	To be tested
TC09	Invalid document upload	Upload an unsupported or corrupted file	Invalid/corrupted file	System should reject the file and display an appropriate error message	To be tested
TC10	AI document classification	Upload a valid legal document and initiate classification	Valid legal document	AI module should classify the document into the appropriate category	To be tested
TC11	Hearing scheduling	Select a case and request a hearing date	Available judge and courtroom	System should suggest/schedule a suitable hearing without conflict	To be tested

Test Case ID	Test Scenario	Test Steps	Test Data	Expected Result	Actual Result
TC12	Schedule conflict	Attempt to schedule two hearings for the same judge and time	Same judge, same time	System should detect the conflict and prevent double booking	To be tested
TC13	Case duration prediction	Provide required case information and request prediction	Valid historical/case data	System should generate an estimated completion timeline	To be tested
TC14	Judicial workload analysis	Open workload dashboard	Existing case records	System should accurately display pending, completed, and assigned cases	To be tested
TC15	Generate analytics report	Select required parameters and generate report	Valid date range and case category	System should generate an accurate analytics report	To be tested
TC16	Notification generation	Create or modify an upcoming hearing	Hearing scheduled for a future date	Relevant authorized users should receive an appropriate notification	To be tested
TC17	Large data processing	Search or process a large number of case records	Large dataset	System should process the request without failure and within an acceptable response time	To be tested

Question 4: Apply Appropriate Test Design Techniques such as Equivalence Partitioning, Boundary Value Analysis, Decision Table, or State Transition Testing

Answer:

Test design techniques help identify effective test cases systematically and reduce the possibility of missing important conditions. For the **AI-Powered Smart Court Case Management and Judicial Analytics Platform**, the following techniques are applicable.

1. Equivalence Partitioning

Equivalence Partitioning divides input data into valid and invalid groups. Instead of testing every possible input, representative values from each group are tested.

Example – Case Number Validation:

- Valid case number → Accepted by the system.
- Invalid/incorrect format → Rejected by the system.
- Empty case number → System should display a validation message.
- Duplicate case number → System should prevent duplicate registration.

Example – User Login:

- Valid username + valid password → Login successful.
- Valid username + invalid password → Login rejected.
- Invalid username + valid password → Login rejected.
- Empty username/password → Validation message displayed.

2. Boundary Value Analysis

Boundary Value Analysis tests values at the minimum and maximum limits because errors frequently occur at boundaries.

Example – Legal Document Upload Size:

Assume the system permits documents up to **10 MB**.

Test Data	Expected Result
9.99 MB	Upload accepted
10 MB	Upload accepted
10.01 MB	Upload rejected
0 MB / empty file	Upload rejected

This ensures that the document size restriction works correctly at and around its boundary.

3. Decision Table Testing

Decision Table Testing is useful when system behavior depends on multiple conditions.

Example – Hearing Schedule:

Judge Available	Courtroom Available	Case Eligible	Expected Result
Yes	Yes	Yes	Hearing can be scheduled
Yes	No	Yes	Scheduling should be prevented
No	Yes	Yes	Scheduling should be prevented
Yes	Yes	No	Scheduling should be prevented
No	No	No	Scheduling should be prevented

This technique ensures that all important combinations of conditions are tested.

4. State Transition Testing

State Transition Testing verifies how the system behaves when a case moves from one state to another.

Example – Court Case Lifecycle:

Registered → Under Review → Hearing Scheduled → Hearing Completed → Judgment Pending → Closed

Test cases should verify that:

- A newly registered case moves to **Registered** status.
- An eligible case can move from **Registered** to **Under Review**.
- A case can move to **Hearing Scheduled** only when scheduling conditions are satisfied.
- After the hearing is completed, the status changes to **Hearing Completed**.
- After the required judicial process, the case can move to **Judgment Pending**.
- A completed case can finally move to **Closed**.
- Unauthorized or invalid state transitions should be rejected.

Conclusion

The combination of **Equivalence Partitioning**, **Boundary Value Analysis**, **Decision Table Testing**, and **State Transition Testing** provides systematic test coverage for the proposed judicial platform. These techniques help identify valid, invalid, boundary, and exceptional conditions and improve the **correctness**, **reliability**, **security**, and **quality** of the system.

Question 5: Establish Traceability and Test Coverage Between Requirements, Test Scenarios, and Test Cases

Answer:

Requirement traceability ensures that every important requirement of the **AI-Powered Smart Court Case Management and Judicial Analytics Platform** is mapped to one or more test cases. It helps verify that all functional and non-functional requirements are tested and that no major requirement is missed.

Requirement Traceability Matrix:

Requirement ID	Requirement	Test Scenario	Test Case ID	Coverage
FR01	User authentication	Verify valid and invalid login	TC01, TC02	Covered
FR02	Role-based access	Verify authorized and unauthorized access	TC03, TC19	Covered
FR03	Case registration	Verify creation of new cases	TC04, TC05	Covered
FR04	Case search	Verify searching and retrieving cases	TC06, TC07	Covered
FR05	Document management	Verify legal document upload and validation	TC08, TC09	Covered
FR06	AI document classification	Verify automatic document classification	TC10	Covered
FR07	Hearing scheduling	Verify hearing scheduling and conflict detection	TC11, TC12	Covered
FR08	Case duration prediction	Verify AI-based case completion prediction	TC13	Covered
FR09	Judicial workload analysis	Verify workload calculation and dashboard	TC14	Covered
FR10	Analytics and reports	Verify generation of judicial reports	TC15	Covered

Requirement ID	Requirement	Test Scenario	Test Case ID	Coverage
FR11	Notifications	Verify hearing and case notifications	TC16	Covered
NFR01	Performance	Verify system response with large datasets	TC17	Covered
NFR02	Reliability	Verify recovery after system failure	TC18, TC20	Covered
NFR03	Security	Verify protection against unauthorized access	TC03, TC19	Covered
NFR04	Data integrity	Verify preservation of case information	TC18, TC20	Covered
NFR05	Usability	Verify clarity and ease of system interaction	TC01, TC04, TC06	Covered

Test Coverage

The test cases provide coverage for the major functional and non-functional requirements of the proposed system. **Functional coverage** includes user authentication, case management, document processing, AI classification, hearing scheduling, case prediction, judicial workload analysis, reporting, and notifications. **Non-functional coverage** includes security, performance, reliability, data integrity, and usability. The test cases also include **normal, invalid, boundary, and exceptional conditions**.

The requirement traceability matrix establishes a clear relationship between **requirements** → **test scenarios** → **test cases** → **expected results**. This helps testers identify missing test cases and ensures that every major requirement is verified.

Therefore, the proposed test suite provides **strong requirement traceability and adequate functional coverage** for the AI-Powered Smart Court Case Management and Judicial Analytics Platform. Any test case that fails during execution should be documented, analyzed, and retested after the defect is corrected.

Question 6: Documentation and Presentation of the Test Case Design

Answer:

The test case design document for the AI-Powered Smart Court Case Management and Judicial Analytics Platform should be organized in a clear, consistent, and professional manner so that developers, testers, court administrators, and other authorized stakeholders can easily understand and execute the test cases.

The document begins with the title of the assessment, course details, student details, and problem statement. It should then present the functional and non-functional requirements that need to be tested. The identified test scenarios should be clearly listed before presenting the detailed test cases.

Each test case should contain a unique Test Case ID, Test Scenario, Test Steps, Test Data, Expected Result, Actual Result, and Pass/Fail Status. This structure ensures that every test can be executed and evaluated systematically. Test cases should cover normal, boundary, invalid, and exceptional conditions.

The document should also clearly explain the test design techniques used, including Equivalence Partitioning, Boundary Value Analysis, Decision Table Testing, and State Transition Testing. The use of these techniques demonstrates systematic and effective test planning.

A Requirement Traceability Matrix should be included to establish a relationship between requirements, test scenarios, and test cases. This ensures that all important requirements are covered and helps identify any gaps in testing.

The test results should be recorded after executing the system. The Actual Result column should contain the observed system behaviour, while the Pass/Fail Status should indicate whether the observed result matches the expected result. Failed test cases should be documented and retested after the corresponding defects are corrected.

Finally, the document should maintain consistent formatting, proper headings, numbered sections, readable tables, correct grammar, and clear technical terminology. Confidential judicial information should not be used as test data; instead, dummy or anonymized data should be used.

Conclusion:

A well-structured test case design document improves the clarity, traceability, completeness, and professionalism of the testing process. It provides evidence that the proposed AI-powered judicial platform has been systematically evaluated for functionality, reliability, security, performance, usability, and ethical use.



Question 7: Overall Test Coverage Summary and Conclusion

Answer:

The AI-Powered Smart Court Case Management and Judicial Analytics Platform has been designed with comprehensive testing coverage to verify its functionality, reliability, security, performance, usability, and ethical considerations. The test cases cover the major requirements of the system, including user authentication, role-based access control, case registration, case searching, legal document management, AI-based document classification, hearing scheduling, case duration prediction, judicial workload analysis, analytics, reports, and notifications.

The testing process includes normal, invalid, boundary, and exceptional conditions. Normal test cases verify that the system performs its intended functions correctly. Invalid test cases verify that incorrect inputs are rejected appropriately. Boundary test cases verify the behavior of the system at minimum and maximum input limits. Exceptional test cases verify that the system handles unexpected situations such as network failures, system interruptions, invalid documents, and unauthorized access safely.

Multiple test design techniques have been applied, including Equivalence Partitioning, Boundary Value Analysis, Decision Table Testing, and State Transition Testing. These techniques help improve test coverage and ensure that different input conditions and system states are properly evaluated.

Requirement traceability is maintained by mapping each requirement to relevant test scenarios and test cases. This ensures that all major functional and non-functional requirements are covered and helps identify gaps in the testing process.

Security and ethical considerations are particularly important because the system handles confidential judicial information and uses AI for classification, prediction, and analytics. Therefore, access control, data protection, transparency of AI-generated results, and the use of anonymized test data should be verified during testing. AI predictions should support judicial administration and should not independently determine judicial decisions.

Conclusion

The proposed test case design provides a systematic approach for evaluating the **AI-Powered Smart Court Case Management and Judicial Analytics Platform**. Comprehensive testing can help ensure that the platform is **accurate, secure, reliable, efficient, transparent, accessible, and user-friendly**. Successful implementation of these test cases will contribute to faster case processing, better judicial workload management, reduced case backlogs, improved transparency, and enhanced legal service quality.